

UNIVERSITAS NEGERI YOGYAKARTA
FAKULTAS TEKNIK
PROGRAM STUDI TEKNIK INDUSTRI

LAPORAN PRAKTIKUM
APLIKASI WEB DAN MOBILE

Semester Genap 2025/2026

**MODUL 15 – Component, Navigation, dan Styling di React Native
+ Proyek Mobile**

Nama Mahasiswa : Nailah Azka Iftinah
NIM : 23051430026
Kelas : A
Tanggal Praktikum : 29 Mei 2026
Dosen Pengampu : Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT.

Dikumpulkan paling lambat 7 hari setelah pelaksanaan praktikum

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul 15 (Pertemuan 15)
Topik Utama	Component, Navigation, dan Styling di React Native + Proyek Mobile
Sub-Topik	React Navigation (Stack Navigator), Passing Data Antar Layar, dan Proyek Inventori Sederhana.
Durasi Praktikum	340 Menit
Tanggal Praktikum	29 Mei 2026
Nama Mahasiswa	Nailah Azka Iftinah
NIM	23051430026
Kelas	A

B. TUJUAN PEMBELAJARAN

Tuliskan tujuan pembelajaran yang ingin dicapai pada modul ini, sesuai dengan yang tertera di Manual Praktikum.

1	Memahami konsep navigasi layar di aplikasi mobile (Stack).
2	Mampu menginstall dan mengonfigurasi `react-navigation`.
3	Membuat aplikasi multi-halaman: Beranda (List) → Detail Item.
4	Meneruskan data (params) dari satu layar ke layar lain.

C. ALAT DAN BAHAN

Sebutkan seluruh perangkat keras, perangkat lunak, dan sumber referensi yang digunakan selama praktikum.

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern (Chrome/Firefox/Edge).	Versi terbaru	Testing & Debugging
3	Ekstensi VS Code	Live Server & CodeSnap	Menjalankan HTML di server lokal dan

			memperbarui tampilan otomatis saat file berubah. Mengubah kode program menjadi gambar untuk dokumentasi
5	Git/Github	Versi Terbaru	Repository dan Manajemen versi
6	JavaScript (Client-Side)	ES6 (Modern JavaScript)	Bahasa pemrograman untuk logika & perhitungan
7	Quokka.js	Extension VS Code	Untuk melihat hasil JavaScript secara realtime
8	Node.js	LTS	Menjalankan react
9	React Router DOM	Versi Terbaru	Mengatur navigasi antar halaman (routing)
10	Fetch API	Built-in browser	Mengambil data dari API eksternal
11	Expo Go	Versi terbaru	Aplikasi di Android/iOS untuk menjalankan project React Native dari QR Code
12	Android Smartphone / iPhone	Android/iOS terbaru	Digunakan sebagai emulator mobile melalui aplikasi Expo Go

D. DASAR TEORI

React Native merupakan framework yang tidak hanya andal dalam menyusun tampilan statis, melainkan juga dalam mengelola alur kerja aplikasi multi-halaman yang kompleks. Pada pertemuan 15 ini, pembelajaran difokuskan pada penerapan React Navigation menggunakan Stack Navigator, mekanisme oper data antar layar (passing data params), serta pengkondisian tampilan (conditional rendering). Berbeda dengan pengembangan berbasis web yang memanfaatkan perubahan URL (seperti /about atau /detail), navigasi pada aplikasi mobile berbasis Android dan iOS menggunakan konsep Screen Stack atau tumpukan layar. Konsep ini bekerja dengan cara menumpuk layar baru di atas layar lama ketika pengguna berpindah halaman, dan membuang (pop) layar teratas dari memori ketika pengguna menekan tombol kembali. Hal ini sangat penting untuk dipahami agar alur kerja pengguna (user workflow) di dalam aplikasi mobile berjalan secara natural, responsif, dan hemat penggunaan memori perangkat.

Konsep utama dalam integrasi navigasi ini adalah multi-screen architecture dan data interchange. Melalui Stack Navigator, beberapa komponen layar mandiri seperti HomeScreen, DetailScreen, dan TambahScreen dapat saling terhubung dalam satu wadah induk bernama NavigationContainer. Perpindahan halaman dieksekusi menggunakan fungsi `navigation.navigate()`, di mana fungsi tersebut tidak hanya memindahkan layar, tetapi juga bertindak sebagai kurir pembawa paket data berupa objek JavaScript (route params). Data yang dikirim dari halaman daftar barang akan diterima di halaman detail menggunakan teknik destructuring `route.params`. Dalam konteks Teknik Industri, kemampuan passing data ini sangat krusial dalam sistem logistik dan manajemen rantai pasok (supply chain management). Operator gudang dapat mengklik satu item spesifik dari ratusan daftar inventori pada HomeScreen, lalu sistem secara instan akan mengoper data SKU, jumlah stok, serta posisi spesifik Lokasi Penyimpanan (Rak/Gudang) barang tersebut ke DetailScreen secara akurat tanpa risiko data tertukar.

Selain masalah navigasi, pada praktikum ini juga dipelajari konsep Conditional Rendering dan visual kontrol (visual control system). Conditional Rendering memungkinkan aplikasi menampilkan komponen tertentu hanya ketika syarat atau kondisi logis tertentu terpenuhi. Sebagai contoh, ketika variabel data stok suatu komponen pabrik bernilai nol, sistem secara otomatis akan memunculkan tombol "Request Stok Darurat". Di dalam dunia industri, logika ini mengadopsi prinsip Just-In-Time (JIT) untuk mencegah terjadinya lini produksi berhenti (production downtime) akibat keterlambatan material. Interaksi data ini dikombinasikan dengan manipulasi status (state management) lokal untuk kebutuhan Quality Control (QC). Ketika seorang inspektur mutu mengubah status kelayakan barang menjadi "Gagal" (Defect) di halaman detail, sistem akan mengirimkan sinyal balik yang memicu komponen FlatList di halaman utama untuk mengubah warna teks item tersebut menjadi merah. Perubahan visual ini berfungsi

sebagai sistem Poka-Yoke atau pencegah kesalahan (Andon), memberikan peringatan instan bagi operator agar tidak mendistribusikan produk cacat ke proses berikutnya. Secara keseluruhan, pertemuan 15 ini memberikan pemahaman mendalam mengenai arsitektur navigasi dan pengelolaan data dinamik pada aplikasi mobile. Dengan menguasai konsep konfigurasi React Navigation, pengiriman parameter antar layar, serta visualisasi kondisi berbasis data riil, mahasiswa tidak hanya mampu membuat aplikasi yang interaktif dari sisi komputasi. Lebih dari itu, mahasiswa dapat merancang sebuah sistem monitoring Quality Control dan manajemen inventori pergudangan yang andal, adaptif, serta mampu meminimalisasi kesalahan input data manusia (human error) di lingkungan industri modern.

D.1 Konsep Utama

Pada praktikum pertemuan kelima belas ini, pembelajaran mulai memasuki pengembangan alur kerja aplikasi mobile yang lebih kompleks menggunakan React Native. Jika pada pertemuan sebelumnya fokus utama masih pada pembuatan dasar tampilan antarmuka (UI) statis tunggal, pada pertemuan ini aplikasi mulai dikembangkan agar memiliki multi-halaman yang saling terhubung melalui sistem navigasi. Dengan pendekatan ini, aplikasi monitoring industri tidak hanya menampilkan informasi pada satu layar saja, melainkan memungkinkan operator gudang atau inspektur mutu untuk menjelajahi berbagai menu seperti daftar barang, detail spesifikasi, hingga form penginputan data baru secara interaktif.

Konsep pertama yang dipelajari adalah Arsitektur Navigasi Mobile Berbasis Screen Stack. Berbeda dengan pengembangan web yang mengandalkan URL text untuk berpindah halaman, React Native menggunakan prinsip tumpukan layar (Stack). Ketika fungsi `navigation.navigate()` dipanggil, layar baru akan ditumpuk di atas layar lama, dan ketika pengguna menekan tombol kembali, fungsi `navigation.goBack()` akan membuang (pop) layar teratas dari memori perangkat. Dalam konteks Teknik Industri, konsep ini sangat membantu untuk menyusun alur kerja (workflow) operator di lapangan secara terstruktur, mulai dari halaman utama inventori hingga masuk ke halaman detail item tertentu.

Konsep kedua adalah Pengiriman Parameter Antar Layar (Passing Data Params). Pada praktikum ini dipelajari bagaimana memindahkan data objek spesifik dari satu halaman ke halaman lainnya secara dinamis. Melalui properti `route.params`, data barang yang dipilih oleh pengguna di HomeScreen dikirimkan sebagai paket data terintegrasi, yang kemudian dibuka (destructuring) oleh DetailScreen untuk menampilkan informasi detailnya. Pendekatan ini memastikan data material, nomor SKU, jumlah stok, hingga posisi lokasi penyimpanan di dalam gudang dapat terintegrasi akurat tanpa risiko data tertukar atau salah input.

Konsep ketiga adalah Conditional Rendering untuk Tombol Darurat. Konsep ini memungkinkan aplikasi untuk menampilkan atau menyembunyikan komponen visual tertentu secara otomatis berdasarkan kondisi logika data yang terpenuhi (if-else statement). Pada praktikum ini, conditional rendering diaplikasikan untuk mendeteksi jumlah stok barang; jika stok bernilai nol, sistem akan otomatis memunculkan tombol tambahan berupa "Request Stok Darurat". Dalam sistem produksi, logika ini mengadopsi prinsip Just-In-Time (JIT) guna mencegah terjadinya keterlambatan material yang bisa memicu berhentinya lini produksi (production downtime).

Konsep keempat adalah State Management untuk Sinkronisasi Kualitas. Pada praktikum ini dipelajari cara memanipulasi status data secara lokal menggunakan React State untuk menjembatani interaksi antarlayar. Ketika data status kelayakan diubah di halaman detail, perubahan tersebut dikirimkan kembali ke halaman utama untuk memicu pembaruan data pada komponen FlatList. Konsep manajemen data ini menjadi fondasi penting dalam membangun sistem pelaporan data yang dinamis, real-time, dan saling tersinkronisasi antara aktivitas inspeksi di lapangan dengan basis data pusat.

Konsep kelima adalah Visual Control System Melalui Dinamis Styling. Konsep ini memanfaatkan perubahan properti StyleSheet secara kondisional berdasarkan status data yang aktif. Pada aplikasi inspeksi Quality Control (QC), ketika suatu item dinyatakan "Gagal" (Defect) oleh inspektur, logika program akan langsung mengubah warna teks atau latar belakang item tersebut di halaman utama menjadi warna merah. Pendekatan ini berfungsi sebagai sistem Poka-Yoke atau Andon visual di era industri modern, memberikan sinyal peringatan instan kepada operator agar tidak memproses atau mendistribusikan produk cacat tersebut.

Konsep keenam adalah Validasi Form Input pada Aplikasi Perkantoran. Pada perancangan layar TambahScreen, dipelajari fungsi kontrol logika untuk memeriksa kelengkapan data input sebelum disimpan ke dalam sistem inventori. Jika pengguna mengosongkan kolom teks wajib, sistem akan memicu jendela pop-up Alert yang membatalkan proses simpan data. Konsep validasi ini sangat krusial di lingkungan industri untuk meminimalisasi faktor kesalahan manusia (human error) serta menjaga keandalan data (data integrity) saat pencatatan arus masuk material gudang (inbound logistics).

Terakhir adalah Integrasi Paket Library React Navigation. Untuk mengaktifkan seluruh fitur perpindahan halaman tersebut, diperlukan instalasi dan konfigurasi dependensi luar seperti @react-navigation/native-stack, react-native-screens, dan react-native-safe-area-context ke dalam lingkungan Expo. Pengaturan kontainer induk NavigationContainer di dalam

file App.js menjadi gerbang utama dalam mendaftarkan seluruh hierarki layar aplikasi mobile. Dengan seluruh konsep tersebut, praktikum pertemuan 15 memberikan dasar penting dalam merancang aplikasi manajemen pergudangan dan kendali mutu berbasis mobile yang andal, adaptif, serta siap mendukung digitalisasi sistem industri modern.

D.2 Keterkaitan dengan Teknik Industri

Jika dikaitkan dengan Teknik Industri, materi pada pertemuan kelima belas ini sangat penting karena memberikan gambaran nyata mengenai digitalisasi alur kerja (*workflow*) pergudangan dan kendali mutu yang dinamis langsung di lapangan. Dari praktikum ini terlihat bahwa industri modern era Manufaktur Cerdas (*Smart Manufacturing*) menuntut pergerakan data yang tidak lagi statis, melainkan bergerak cepat secara *real-time* mengikuti mobilitas tinggi supervisor dan operator. Dengan adanya arsitektur navigasi multi-halaman pada aplikasi mobile, proses pelaporan deviasi kualitas produk maupun mutasi material dapat diselesaikan dalam hitungan detik langsung dari area lantai produksi tanpa harus menunda pekerjaan untuk kembali ke komputer kantor.

Penerapan sistem *Screen Stack* dan *Passing Data Params* pada praktikum ini merepresentasikan konsep ketertelusuran produk (*product traceability*) dan integrasi aliran informasi dalam manajemen rantai pasok (*supply chain management*). Ketika operator memindahkan layar dari daftar inventori menuju halaman detail barang, data krusial seperti nomor SKU, jumlah stok, hingga informasi spesifik Lokasi Penyimpanan (Rak/Gudang) ikut terbawa secara otomatis. Hal ini sangat relevan dengan prinsip efisiensi ergonomi kognitif di lingkungan industri, di mana eliminasi langkah penginputan ulang data manual (*re-entry data*) secara efektif mampu memangkas waktu proses (*lead time*) serta mengeliminasi potensi kesalahan manusia (*human error*) akibat salah ketik kode komponen oleh operator.

Selain itu, penerapan *Conditional Rendering* untuk memunculkan tombol "Request Stok Darurat" saat jumlah persediaan habis bernilai nol mengadopsi prinsip sistem tarik (*Pull System*) dalam manajemen inventori *Just-In-Time* (JIT). Konsep ini memberikan pemahaman berharga bahwa sebuah aplikasi industri harus bersifat prediktif dan mampu memicu tindakan korektif (*corrective action*) secara instan guna mencegah terjadinya berhentinya lini produksi (*production downtime*). Pengaturan tata letak responsif menggunakan Flexbox dan komponen gulir juga menjamin visualisasi data logistik yang padat dapat dikonsumsi dengan nyaman oleh pengguna, baik melalui perangkat *smartphone* operasional berukuran kecil hingga tablet industri yang digunakan di gudang.

Interaksi status inspeksi *Quality Control* (QC) dengan sistem kontrol visual (*Visual Control System*) dinamis (teks berubah merah saat status "Gagal") merupakan perwujudan digital dari konsep *Poka-Yoke* (pencegah kesalahan) dan sistem *Andon*. Di dalam pabrik modern, visualisasi anomali atau produk cacat (*defect*) harus disampaikan secara mencolok agar operator gudang langsung mendapatkan sinyal peringatan dini (*early warning*) untuk mengisolasi barang tersebut agar tidak terdistribusi ke proses berikutnya. Secara keseluruhan, praktikum pertemuan 15 ini memberikan pemahaman bahwa perkembangan teknologi di bidang Teknik Industri tidak hanya berfokus pada mesin, tetapi juga pada pengembangan aplikasi digital yang mampu membantu pengawasan dan pengambilan keputusan secara *real-time* langsung dari perangkat mobile.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

Dokumentasikan setiap langkah kerja yang Anda lakukan, beserta hasil yang diperoleh (screenshot, kode, atau output). Jawab sesuai urutan langkah di Manual Praktikum.

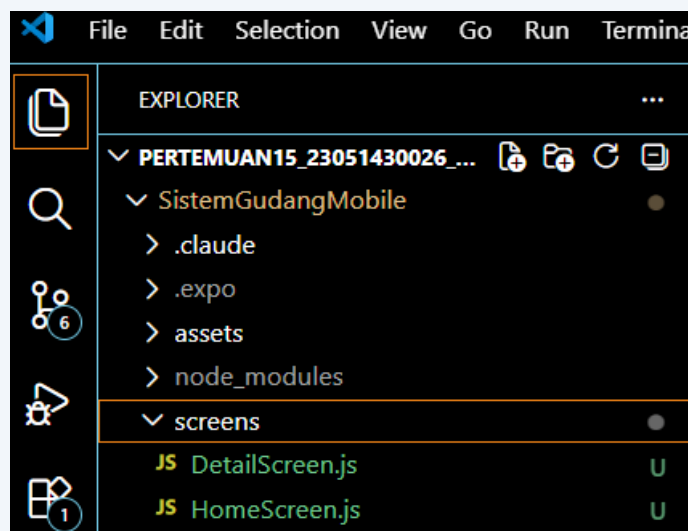
E.1 Langkah 1: Struktur Folder

Buat folder baru di dalam project bernama `screens`. Di dalamnya buat dua file:

1. `HomeScreen.js` (Daftar Inventori)
2. `DetailScreen.js` (Detail Barang)

```
/* ===== KODE / OUTPUT LANGKAH 1=====
1. Folder Baru 'SistemGudangMobile'
2. File `HomeScreen.js` (Daftar Inventori)
3. File `DetailScreen.js` (Detail Barang)
```

Screenshot Hasil:



Gambar E.1– Hasil Membuat Struktur Folder

E.2 Langkah 2: Membuat HomeScreen (Daftar Inventori)

Buka `screens/HomeScreen.js`.

```
/* ===== KODE / OUTPUT LANGKAH 2=====
import React from 'react';
import { View, Text, FlatList, TouchableOpacity, StyleSheet } from 'react-native';

const DATA_INVENTORI = [
  { id: '1', nama: 'Baut M10', stok: 500, lokasi: 'Rak A-1' },
  { id: '2', nama: 'Oli Mesin 20L', stok: 12, lokasi: 'Rak B-3' },
  { id: '3', nama: 'Packing Kayu', stok: 100, lokasi: 'Gudang Luar' },
  { id: '4', nama: 'Mur Ring 12', stok: 0, lokasi: 'Rak A-2' },
];

function HomeScreen({ navigation }) {
  const renderItem = ({ item }) => (
    <TouchableOpacity
      style={styles.itemContainer}
      onPress={() => navigation.navigate('Detail', { itemData: item })}
    >
      <Text style={styles.itemTitle}>{item.nama}</Text>
      <View style={styles.itemInfo}>
        <Text style={styles.itemSub}>Stok: {item.stok}</Text>
        <Text style={styles.itemSub}>{item.lokasi}</Text>
      </View>
    </TouchableOpacity>
  );

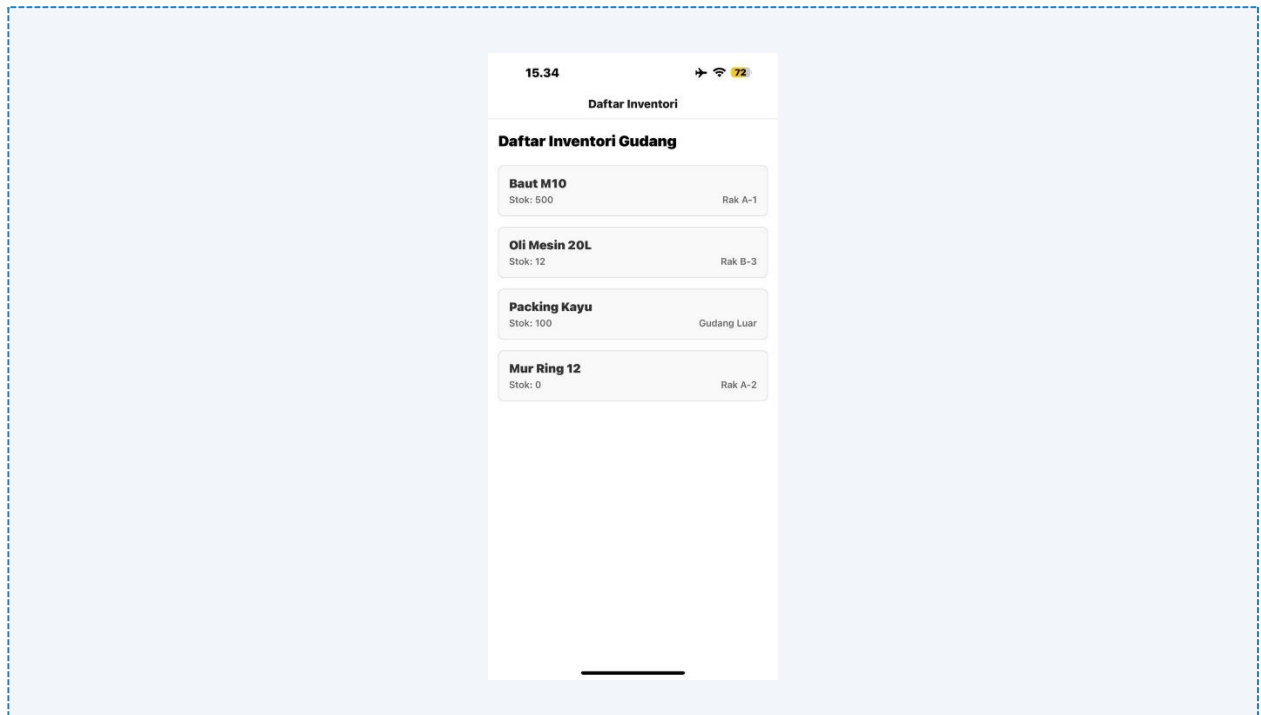
  return (
    <View style={styles.container}>
      <Text style={styles.header}>Daftar Inventori Gudang</Text>
      <FlatList
        data={DATA_INVENTORI}
        renderItem={renderItem}
        keyExtractor={item => item.id}
      />
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    paddingTop: 20,
  },
});
```

```
header: {
  fontSize: 22,
  fontWeight: 'bold',
  marginBottom: 15,
  paddingHorizontal: 15,
},
itemContainer: {
  backgroundColor: '#f9f9f9',
  padding: 15,
  marginVertical: 8,
  marginHorizontal: 15,
  borderRadius: 8,
  borderWidth: 1,
  borderColor: '#ddd',
},
itemTitle: {
  fontSize: 18,
  fontWeight: 'bold',
  color: '#333',
},
itemInfo: {
  flexDirection: 'row',
  justifyContent: 'space-between',
  marginTop: 5,
},
itemSub: {
  color: '#666',
},
});

export default HomeScreen;
```

Screenshot Hasil:



Gambar E.2 – Hasil Membuat HomeScreen (Daftar Inventori)

E.3 Langkah 3: Membuat DetailScreen

Buka `screens/DetailScreen.js`.

```
import React from 'react';
import { View, Text, StyleSheet, Button } from 'react-native';

function DetailScreen({ route, navigation }) {
  const { itemData } = route.params;

  return (
    <View style={styles.container}>
      <View style={styles.card}>
        <Text style={styles.label}>Nama Barang:</Text>
        <Text style={styles.value}>{itemData.nama}</Text>

        <Text style={styles.label}>Stok Saat Ini:</Text>
        <Text style={styles.value}>{itemData.stok} Unit</Text>

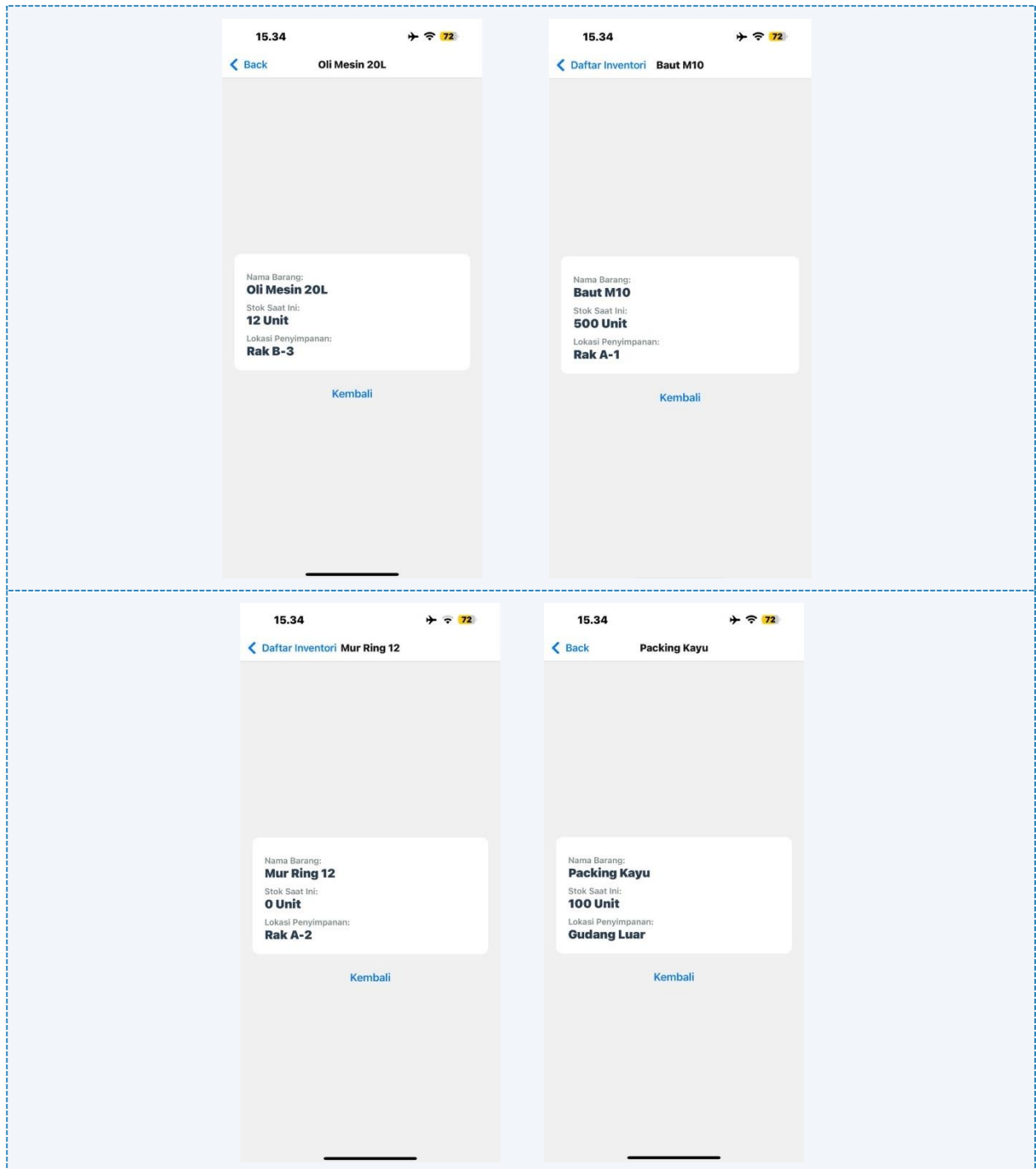
        <Text style={styles.label}>Lokasi Penyimpanan:</Text>
        <Text style={styles.value}>{itemData.lokasi}</Text>
      </View>
      <Button title="Kembali" onPress={() => navigation.goBack()} />
    </View>
  );
}
```

```
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: 'center',
    padding: 20,
    backgroundColor: '#f0f0f0',
  },
  card: {
    backgroundColor: 'white',
    padding: 20,
    borderRadius: 10,
    marginBottom: 20,
    elevation: 3,
  },
  label: {
    fontSize: 14,
    color: '#7f8c8d',
    marginTop: 10,
  },
  value: {
    fontSize: 20,
    fontWeight: 'bold',
    color: '#2c3e50',
  },
  dangerText: {
    color: 'red',
  },
  successText: {
    color: 'green',
  }
});

export default DetailScreen;
```

Screenshot Hasil:



Gambar E.3 – Hasil Membuat DetailScreen

E.4 Langkah 4: Menghubungkan Navigation di App.js

Buka file utama `App.js` dan modifikasi menjadi sebagai berikut:

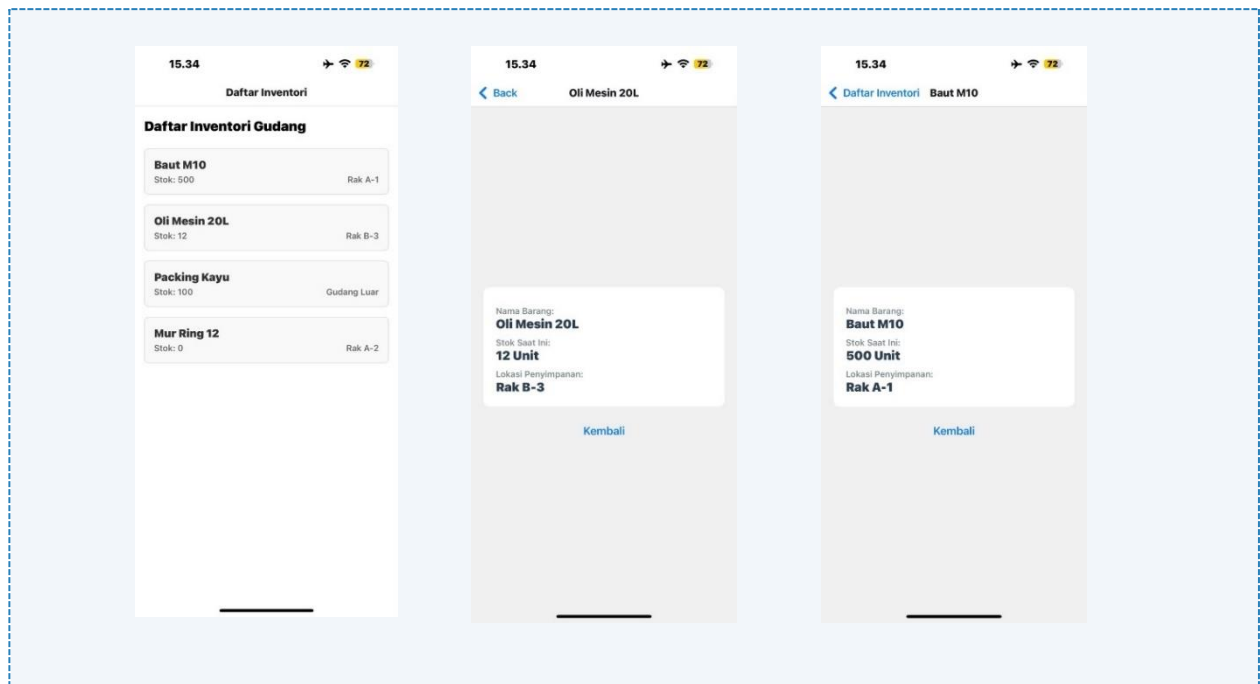
```
/* ===== KODE / OUTPUT LANGKAH 2=====  
import React from 'react';  
import { NavigationContainer } from '@react-navigation/native';  
import { createNativeStackNavigator } from '@react-navigation/native-stack';
```

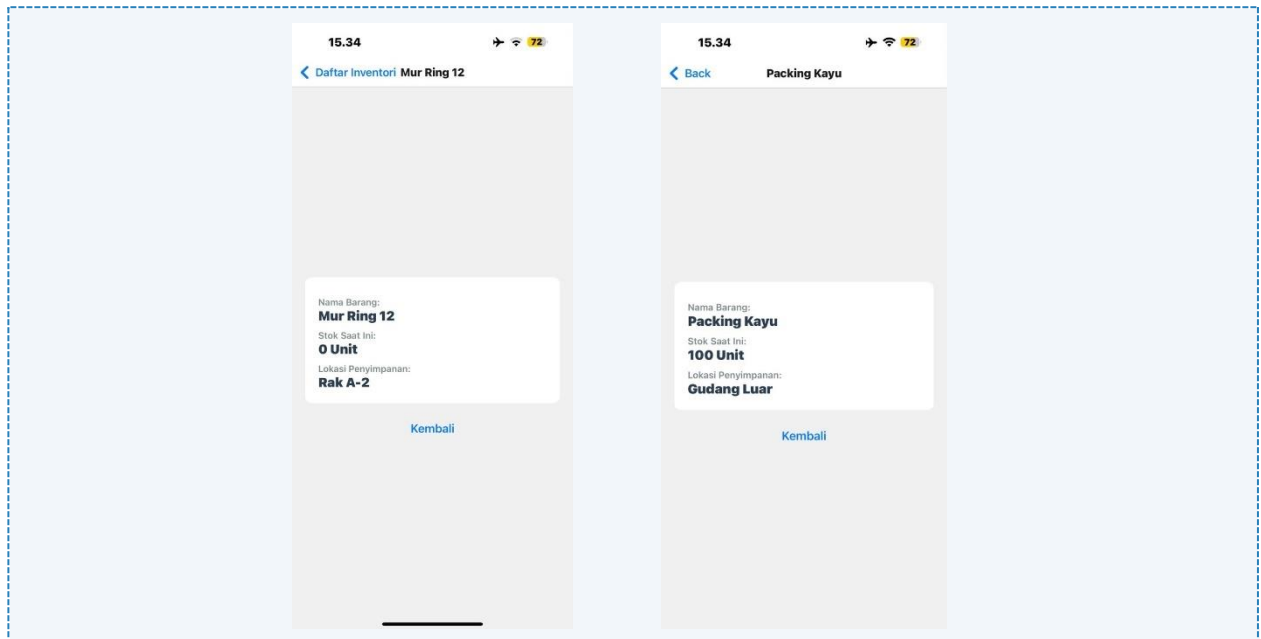
```
import HomeScreen from './screens/HomeScreen';
import DetailScreen from './screens/DetailScreen';

const Stack = createNativeStackNavigator();

export default function App() {
  return (
    <NavigationContainer>
      <Stack.Navigator>
        <Stack.Screen
          name="Home"
          component={HomeScreen}
          options={{ title: 'Daftar Inventori' }}
        />
        <Stack.Screen
          name="Detail"
          component={DetailScreen}
          options={({ route }) => ({ title: route.params?.itemData.nama ||
'Detail' })}
        />
      </Stack.Navigator>
    </NavigationContainer>
  );
}
```

Screenshot Hasil:





Gambar E.4 – Hasil Menghubungkan Navigation di App.js

F. LATIHAN DAN TUGAS MANDIRI

Jawab dan dokumentasikan seluruh latihan/tugas yang ada di bagian "Latihan/Tugas Mandiri" pada modul Anda.

F.1 Latihan 1: (Conditional Rendering)

Pertanyaan / Instruksi Latihan:

Latihan 1 (Conditional Rendering): Pada `DetailScreen`, jika stok = 0, munculkan tombol "Request Stok Darurat" (gunakan Button tambahan).

Jawaban / Kode Solusi:

```
import React from 'react';
import { View, Text, StyleSheet, Button, Alert } from 'react-native';

function DetailScreen({ route, navigation }) {
  const { itemData } = route.params;

  return (
    <View style={styles.container}>
      <View style={styles.card}>
        <Text style={styles.label}>Nama Barang:</Text>
        <Text style={styles.value}>{itemData.nama}</Text>

        <Text style={styles.label}>Stok Saat Ini:</Text>
        <Text style={[[styles.value, itemData.stok === 0 ? styles.dangerText :
styles.successText]]}>
          {itemData.stok} Unit
        </Text>
      </View>
    </View>
  );
}
```

```
        </Text>

        <Text style={styles.label}>Lokasi Penyimpanan:</Text>
        <Text style={styles.value}>{itemData.lokasi}</Text>
    </View>

    {itemData.stok === 0 && (
        <View style={styles.btnAction}>
            <Button
                title="Request Stok Darurat"
                color="red"
                onPress={() => Alert.alert("Sistem Logistik", "Permintaan stok
darurat telah dikirim!")}
            />
        </View>
    )}

    <Button title="Kembali" onPress={() => navigation.goBack()} />
</View>
);
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: 'center',
    padding: 20,
    backgroundColor: '#f0f0f0',
  },
  card: {
    backgroundColor: 'white',
    padding: 20,
    borderRadius: 10,
    marginBottom: 20,
    elevation: 3,
  },
  label: {
    fontSize: 14,
    color: '#7f8c8d',
    marginTop: 10,
  },
  value: {
    fontSize: 20,
    fontWeight: 'bold',
    color: '#2c3e50',
  },
});
```



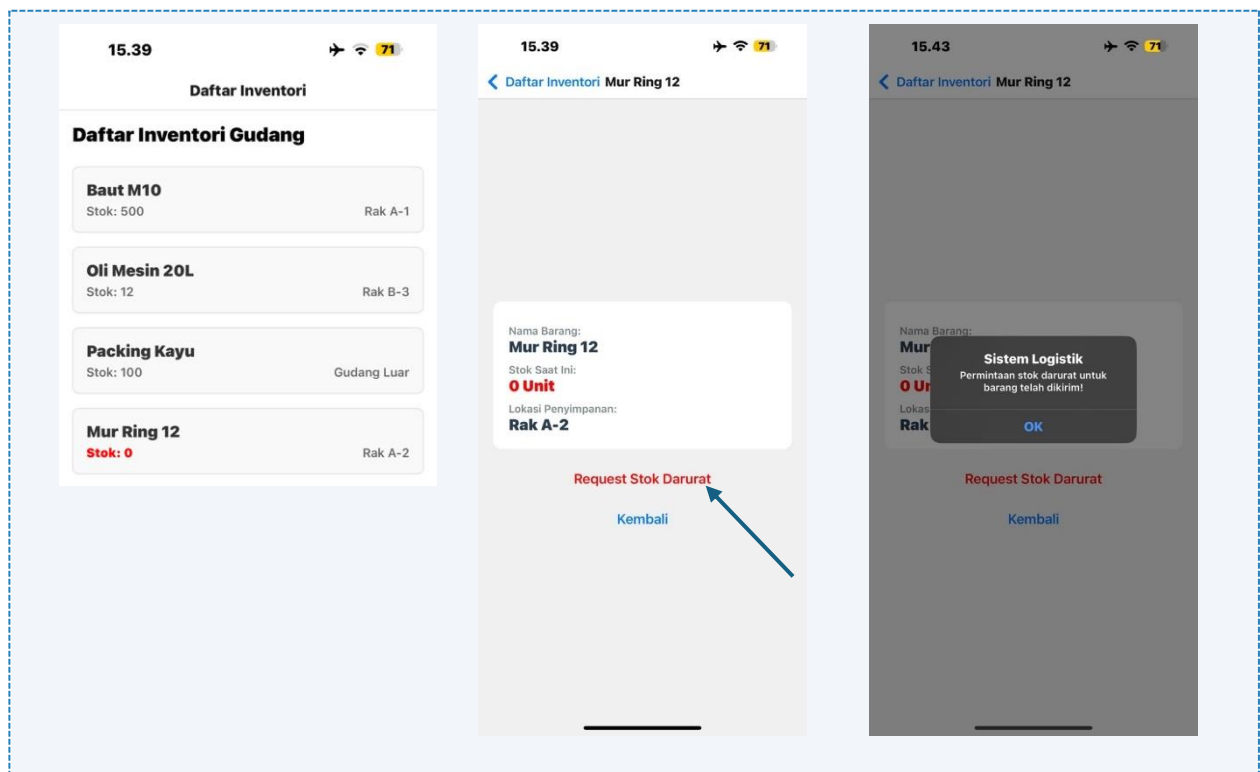
```

dangerText: {
  color: 'red',
},
successText: {
  color: 'green',
},
},
btnAction: {
  marginBottom: 15,
},
});

export default DetailScreen;

```

Screenshot Hasil:



Gambar F.1 – Hasil Latihan 1

F.2 Latihan 2 : (Passing Data)

Pertanyaan / Instruksi Latihan:

Latihan 2 (Passing Data): Di `HomeScreen`, tambahkan tombol FAB (Floating Action Button) atau Button di bawah untuk "Tambah Barang". Saat ditekan, navigasi ke layar (screen baru bernama `TambahScreen`) yang berisi form kosong.

Jawaban / Kode Solusi:

1. JAWABAN KODE Latihan 2 (HomeScreen.js)

```
import React from 'react';
import { View, Text, FlatList, TouchableOpacity, StyleSheet, ScrollView }
from 'react-native';

const DATA_INVENTORI = [
  { id: '1', nama: 'Baut M10', stok: 500, lokasi: 'Rak A-1' },
  { id: '2', nama: 'Oli Mesin 20L', stok: 12, lokasi: 'Rak B-3' },
  { id: '3', nama: 'Packing Kayu', stok: 100, lokasi: 'Gudang Luar' },
  { id: '4', nama: 'Mur Ring 12', stok: 0, lokasi: 'Rak A-2' },
];

function HomeScreen({ navigation }) {
  const renderItem = ({ item }) => (
    <TouchableOpacity
      style={styles.itemContainer}
      onPress={() => navigation.navigate('Detail', { itemData: item })}
    >
      <Text style={styles.itemTitle}>{item.nama}</Text>
      <View style={styles.itemInfo}>
        <Text style={[styles.itemSub, item.stok === 0 && styles.dangerText]}>
          Stok: {item.stok}
        </Text>
        <Text style={styles.itemSub}>{item.lokasi}</Text>
      </View>
    </TouchableOpacity>
  );

  return (
    <ScrollView style={styles.container}>
      <Text style={styles.header}>Daftar Inventori Gudang</Text>
      <FlatList
        data={DATA_INVENTORI}
        renderItem={renderItem}
        keyExtractor={item => item.id}
        scrollEnabled={false}
      />
      <TouchableOpacity
        style={styles.btnTambah}
        onPress={() => navigation.navigate('Tambah')}
      >
        <Text style={styles.btnTambahText}>+ Tambah Barang</Text>
      </TouchableOpacity>
    </ScrollView>
  );
}
```

```
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    padding: 15,
  },
  header: {
    fontSize: 22,
    fontWeight: 'bold',
    marginBottom: 15,
  },
  itemContainer: {
    backgroundColor: '#f9f9f9',
    padding: 15,
    marginVertical: 8,
    borderRadius: 8,
    borderWidth: 1,
    borderColor: '#ddd',
  },
  itemTitle: {
    fontSize: 18,
    fontWeight: 'bold',
    color: '#333',
  },
  itemInfo: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    marginTop: 5,
  },
  itemSub: {
    color: '#666',
  },
  dangerText: {
    color: 'red',
    fontWeight: 'bold',
  },
  btnTambah: {
    backgroundColor: '#007bff',
    padding: 15,
    borderRadius: 8,
    alignItems: 'center',
    marginTop: 20,
    marginBottom: 30,
  },
});
```

```
    btnTambahText: {
      color: '#fff',
      fontWeight: 'bold',
      fontSize: 16,
    },
  });

export default HomeScreen;
```

2. JAWABAN KODE Latihan 2 (TambahScreen.js)

```
import React from 'react';
import { View, Text, TextInput, StyleSheet, Button } from 'react-native';

function TambahScreen({ navigation }) {
  return (
    <View style={styles.container}>
      <Text style={styles.title}>Form Tambah Barang Baru</Text>

      <Text style={styles.label}>Nama Barang</Text>
      <TextInput style={styles.input} placeholder="Nama komponen..." />

      <Text style={styles.label}>Stok Awal</Text>
      <TextInput style={styles.input} placeholder="0" keyboardType="numeric"
    />

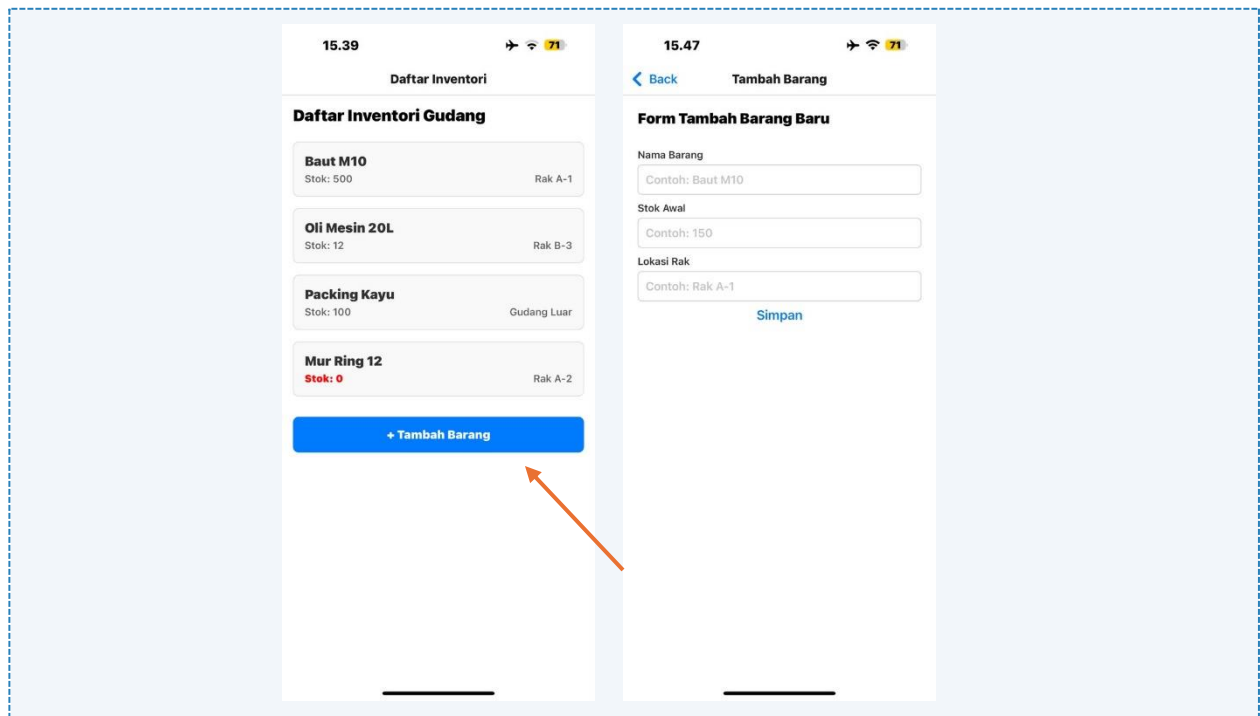
      <Text style={styles.label}>Lokasi Rak</Text>
      <TextInput style={styles.input} placeholder="Lokasi..." />

      <Button title="Simpan" onPress={() => navigation.goBack()} />
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 20,
    backgroundColor: '#fff',
  },
  title: {
    fontSize: 20,
    fontWeight: 'bold',
    marginBottom: 20,
  },
  label: {
```

```
fontSize: 14,  
color: '#333',  
marginTop: 10,  
marginBottom: 5,  
},  
input: {  
  borderWidth: 1,  
  borderColor: '#ccc',  
  padding: 10,  
  borderRadius: 6,  
  fontSize: 16,  
},  
});  
  
export default TambahScreen;
```

Screenshot Hasil:



Gambar F.2 – Hasil Latihan 2

F.4 Tugas Proyek Mini (Aplikasi Inspeksi QC):

Deskripsi proyek mini:

Buat aplikasi inspeksi sederhana.

a. Home: Daftar item yang perlu diinspeksi.

b. Detail: Foto item, standar kualitas, dan dropdown status (Lolos/Gagal).

c. Simulasi: Saat user memilih "Gagal" di layar detail dan kembali, ubah warna teks di Home menjadi merah (memerlukan sedikit logika state management sederhana).

Penjelasan pendekatan/solusi yang digunakan:

Pada proyek mini pertemuan 15 ini, pendekatan yang digunakan berfokus pada pengembangan aplikasi sistem inspeksi Quality Control (QC) berbasis React Native dengan alur kerja multi-halaman yang lebih dinamis, interaktif, dan mudah dipahami oleh pengguna. Aplikasi dirancang menggunakan komponen utama dari library React Navigation seperti `NavigationContainer` dan `createNativeStackNavigator` untuk membangun infrastruktur perpindahan layar, sedangkan halaman `HomeScreen` dan `DetailScreen` digunakan sebagai struktur utama layout aplikasi. Pada tahap awal, dilakukan penyusunan struktur navigasi tumpukan (`Screen Stack`) di mana halaman beranda dikonfigurasi sebagai gerbang awal untuk menampilkan daftar item produksi yang perlu diinspeksi oleh operator, sementara halaman detail dipersiapkan untuk memuat parameter uji mutu produk secara spesifik.

Selanjutnya, diterapkan konsep arsitektur data terintegrasi dan passing data params pada React Native untuk memberikan fungsionalitas monitoring industri yang lebih profesional. Setiap item barang pada halaman utama memanfaatkan komponen `TouchableOpacity` agar dapat ditekan, yang kemudian memicu fungsi navigasi untuk mengoper paket data material menuju halaman detail. Pada bagian `DetailScreen`, informasi item ditampilkan secara lengkap meliputi foto produk, indikator standar kualitas, serta pilihan dropdown status kelayakan berupa opsi Lolos atau Gagal. Keunggulan simulasi pada proyek ini terletak pada rekayasa logika state management sederhana; ketika inspektur memilih opsi status "Gagal" di layar detail dan kembali ke beranda, sistem secara otomatis akan merespons dengan mengubah warna teks komponen barang tersebut di halaman utama menjadi warna merah sebagai indikator visual anomali (`Visual Control System`).

Selain fokus pada sinkronisasi data, proyek ini juga menerapkan konsep arsitektur `component-based` dan sistem `Poka-Yoke` digital sehingga struktur aplikasi menjadi lebih aman dari risiko kesalahan data (`human error`). Penggunaan fungsi pembatasan seperti tombol `Request Stok Darurat` yang muncul secara kondisional saat stok bernilai nol, serta pemberian warna merah sebagai sinyal peringatan dini (`Andon`), membantu proses pengawasan mutu barang di lantai pabrik menjadi lebih praktis dan sederhana. Secara keseluruhan, proyek mini pada pertemuan 15 ini merupakan implementasi dari materi React Native yang telah dipelajari, mulai dari konfigurasi `Stack Navigation`, mekanisme oper data antar layar, `conditional rendering`, hingga manipulasi status data interaktif. Hasil akhirnya adalah aplikasi sistem kendali mutu berbasis mobile yang andal, responsif, dan memiliki fungsionalitas yang mendekati sistem inspeksi digital pada dunia industri modern.

Kode Utama:

1. Jawaban Kode Mini Proyek (HomeScreen.js)

```
import React, { useState, useEffect } from 'react';
import { View, Text, FlatList, TouchableOpacity, StyleSheet, SafeAreaView, Image }
from 'react-native';

const INITIAL_QC_DATA = [
  {
    id: '1',
    nama: 'Body Panel Depan',
    kode: 'BP-001',
    status: 'Pending',
    gambar:
'https://res.cloudinary.com/demo/image/upload/w_1000,c_fill,ar_16:9,g_auto/car.jpg',
    standar: [
      'Bebas dari penyok, baret, atau cacat cetakan struktural.',
      'Ketebalan lapisan cat merata sesuai batas toleransi mil ( $\mu$ m).',
      'Keselarasan celah (gap) antar-panel maksimal 1.5mm.'
    ]
  },
  {
    id: '2',
    nama: 'Poros Transmisi',
    kode: 'PT-042',
    status: 'Pending',
    gambar: 'https://images.unsplash.com/photo-1619642751034-765dfdf7c58e?auto=format&fit=crop&q=80&w=600',
    standar: [
      'Presisi diameter dan kelurusan poros maksimun toleransi +/- 0.02mm.',
      'Bebas dari retak rambut internal (lolos uji Ultrasonic Testing).',
      'Kekerasan material memenuhi standar spesifikasi Rockwell (HRC).'
    ]
  },
  {
    id: '3',
    nama: 'Kelistrikan ECU',
    kode: 'EC-099',
    status: 'Pending',
    gambar: 'https://images.unsplash.com/photo-1555664424-778a1e5e1b48?auto=format&fit=crop&q=80&w=600',
    standar: [
      'Nilai hambatan dan kontinuitas sirkuit 100% normal.',
      'Pin konektor lurus sempurna tanpa ada risiko korsleting.',
      'Lolos pengujian simulasi firmware dan pembacaan sensor.'
    ]
  }
],
```

```
{
  id: '4',
  nama: 'Sistem Pengereman',
  kode: 'BR-015',
  status: 'Pending',
  gambar: 'https://images.unsplash.com/photo-1511919884226-fd3cad34687c?auto=format&fit=crop&q=80&w=600',
  standar: [
    'Ketebalan piringan cakram (rotor) merata tanpa gelombang.',
    'Bebas kebocoran minyak pada segel caliper dan slang rem.',
    'Kampas rem terpasang presisi dengan koefisien gesek standar.'
  ]
},
];

function HomeScreen({ navigation, route }) {
  const [items, setItems] = useState(INITIAL_QC_DATA);

  useEffect(() => {
    if (route.params?.updatedId) {
      setItems(prevItems =>
        prevItems.map(item =>
          item.id === route.params.updatedId ? { ...item, status:
route.params.status } : item
        )
      );
      navigation.setParams({ updatedId: undefined, status: undefined });
    }
  }, [route.params?.updatedId, route.params?.status]);

  useEffect(() => {
    if (route.params?.newBarang) {
      setItems(prevItems => [...prevItems, route.params.newBarang]);
      navigation.setParams({ newBarang: undefined });
    }
  }, [route.params?.newBarang]);

  const renderItem = ({ item }) => (
    <TouchableOpacity
      style={styles.card}
      onPress={() => navigation.navigate('Detail', { itemData: item })}
    >
    <View style={styles.cardContent}>
      <View>
        <Text style={styles.itemKode}>{item.kode}</Text>

```



```

        <Text style={[styles.itemNama, item.status === 'Gagal' &&
styles.textGagal]}>
            {item.nama}
        </Text>
    </View>
    <View style={[styles.statusBadge, item.status === 'Lolos' ?
styles.badgeLolos : item.status === 'Gagal' ? styles.badgeGagal :
styles.badgePending]}>
        <Text style={styles.statusText}>{item.status}</Text>
    </View>
</View>
</TouchableOpacity>
);

return (
    <SafeAreaView style={styles.container}>
        <View style={styles.headerBox}>
            <View style={styles.titleContainer}>
                <Image
                    source={require('../assets/Logo Teknik Industri UNY.png')}
                    style={styles.logoApp}
                />
                <Text style={styles.headerTitle}>QC INSPECTION SYSTEM</Text>
            </View>
            <View style={styles.identityBar}>
                <Text style={styles.identityText}>Inspector: Nailah Azka Iftinah |
23051430026</Text>
            </View>
        </View>

        <View style={styles.actionContainer}>
            <Text style={styles.sectionHeading}>Daftar Inventaris QC</Text>
            <TouchableOpacity style={styles.btnAddItem} onPress={() =>
navigation.navigate('Tambah')}>
                <Text style={styles.btnAddItemText}>+ BARANG</Text>
            </TouchableOpacity>
        </View>

        <FlatList
            data={items}
            renderItem={renderItem}
            keyExtractor={item => item.id}
            contentContainerStyle={styles.listPadding}
        />
    </SafeAreaView>
);

```

```

}

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: '#f4f7f6' },
  headerBox: { backgroundColor: '#1e293b', padding: 20, paddingTop: 40 },
  titleContainer: { flexDirection: 'row', alignItems: 'center', justifyContent:
'center', gap: 10 },
  logoApp: { width: 40, height: 40, resizeMode: 'contain' },
  headerTitle: { color: '#fff', fontSize: 20, fontWeight: 'bold' },
  identityBar: { backgroundColor: '#334155', marginTop: 12, padding: 8,
borderRadius: 5 },
  identityText: { color: '#cbd5e1', fontSize: 12, textAlign: 'center', fontWeight:
'600' },
  actionContainer: { flexDirection: 'row', justifyContent: 'space-between',
alignItems: 'center', paddingHorizontal: 20, marginTop: 20, marginBottom: 5 },
  sectionHeading: { fontSize: 15, fontWeight: 'bold', color: '#475569',
textTransform: 'uppercase' },
  btnAddItem: { backgroundColor: '#3b82f6', paddingHorizontal: 12, paddingVertical:
6, borderRadius: 6 },
  btnAddItemText: { color: '#fff', fontWeight: 'bold', fontSize: 12 },
  listPadding: { padding: 15, paddingTop: 10 },
  card: { backgroundColor: '#fff', borderRadius: 12, padding: 18, marginBottom: 12,
elevation: 3 },
  cardContent: { flexDirection: 'row', justifyContent: 'space-between', alignItems:
'center' },
  itemKode: { fontSize: 12, color: '#64748b', fontWeight: 'bold' },
  itemNama: { fontSize: 17, fontWeight: 'bold', color: '#334155', marginTop: 2 },
  textGagal: { color: '#ef4444' },
  statusBadge: { paddingHorizontal: 12, paddingVertical: 6, borderRadius: 20,
minWidth: 70, alignItems: 'center' },
  statusText: { color: '#fff', fontSize: 12, fontWeight: 'bold' },
  badgePending: { backgroundColor: '#94a3b8' },
  badgeLolos: { backgroundColor: '#22c55e' },
  badgeGagal: { backgroundColor: '#ef4444' },
});

export default HomeScreen;

```

2. Jawaban Kode Mini Proyek (DetailScreen.js)

```

import React, { useState } from 'react';
import { View, Text, Image, StyleSheet, TouchableOpacity, ScrollView } from 'react
native';

function DetailScreen({ route, navigation }) {
  const { itemData } = route.params;

```

```

const [selectedStatus, setSelectedStatus] = useState(itemData.status);

const handleFinish = () => {
  navigation.navigate('Home', {
    updatedId: itemData.id,
    status: selectedStatus
  });
};

return (
  <ScrollView style={styles.container}>
    <Image
      source={{ uri: itemData.gambar }}
      style={styles.itemImage}
    />

    <View style={styles.content}>
      <Text style={styles.sectionTitle}>Item Informasi</Text>
      <View style={styles.infoRow}>
        <Text style={styles.label}>Nama:</Text>
        <Text style={styles.value}>{itemData.nama}</Text>
      </View>
      <View style={styles.infoRow}>
        <Text style={styles.label}>Serial:</Text>
        <Text style={styles.value}>{itemData.kode}</Text>
      </View>

      <View style={styles.divider} />

      <Text style={styles.sectionTitle}>Standar Kualitas Teknis</Text>
      {itemData.standar.map((std, index) => (
        <Text key={index} style={styles.standardText}>• {std}</Text>
      ))}

      <View style={styles.divider} />

      <Text style={styles.sectionTitle}>Keputusan Inspeksi</Text>
      <View style={styles.btnRow}>
        <TouchableOpacity
          style={[styles.btnAction, selectedStatus === 'Lolos' &&
styles.btnActiveLolos]}
          onPress={() => setSelectedStatus('Lolos')}
        >
          <Text style={[styles.btnText, selectedStatus === 'Lolos' &&
styles.textWhite]}>LOLOS</Text>
        </TouchableOpacity>
      </View>
    </View>
  </ScrollView>
);

```

```

        <TouchableOpacity
          style={[styles.btnAction, selectedStatus === 'Gagal' &&
styles.btnActiveGagal]}
          onPress={() => setSelectedStatus('Gagal')}
        >
          <Text style={[styles.btnText, selectedStatus === 'Gagal' &&
styles.textWhite]}>GAGAL</Text>
        </TouchableOpacity>
      </View>

      <TouchableOpacity style={styles.btnSubmit} onPress={handleFinish}>
        <Text style={styles.btnSubmitText}>SIMPAN HASIL INSPEKSI</Text>
      </TouchableOpacity>
    </View>
  </ScrollView>
);
}

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: '#fff' },
  itemImage: { width: '100%', height: 250, backgroundColor: '#e2e8f0' },
  content: { padding: 20 },
  sectionTitle: { fontSize: 14, fontWeight: 'bold', color: '#1e293b', marginBottom:
10, textTransform: 'uppercase' },
  infoRow: { flexDirection: 'row', marginBottom: 5 },
  label: { width: 60, color: '#64748b', fontWeight: '600' },
  value: { color: '#1e293b', fontWeight: 'bold', fontSize: 16 },
  divider: { height: 1, backgroundColor: '#e2e8f0', marginVertical: 20 },
  standardText: { color: '#475569', fontSize: 15, marginBottom: 8, lineHeight: 22 },
  btnRow: { flexDirection: 'row', gap: 10, marginTop: 10 },
  btnAction: { flex: 1, padding: 15, borderRadius: 8, borderWidth: 2, borderColor:
'#e2e8f0', alignItems: 'center' },
  btnActiveLolos: { backgroundColor: '#22c55e', borderColor: '#22c55e' },
  btnActiveGagal: { backgroundColor: '#ef4444', borderColor: '#ef4444' },
  btnText: { fontWeight: 'bold', color: '#64748b' },
  textWhite: { color: '#fff' },
  btnSubmit: { backgroundColor: '#1e293b', padding: 18, borderRadius: 10,
alignItems: 'center', marginTop: 30 },
  btnSubmitText: { color: '#fff', fontWeight: 'bold', fontSize: 16 },
});

export default DetailScreen;

```

3. Jawaban Kode Mini Proyek (TambahanScreen.js)

```
import React, { useState } from 'react';
import { View, Text, TextInput, StyleSheet, TouchableOpacity, Alert } from 'react-native';

function TambahScreen({ navigation }) {
  const [nama, setNama] = useState('');
  const [kode, setKode] = useState('');
  const [lokasi, setLokasi] = useState('');

  const handleSimpan = () => {
    if (nama.trim() === '' || kode.trim() === '' || lokasi.trim() === '') {
      Alert.alert(
        "Form Belum Lengkap",
        "Harus isi semua data terlebih dahulu baru bisa simpan form!",
        [{ text: "OK" }]
      );
      return;
    }

    navigation.navigate('Home', {
      newBarang: {
        id: Math.random().toString(),
        nama: nama,
        kode: kode.toUpperCase(),
        status: 'Pending',
        gambar: 'https://images.unsplash.com/photo-1537462715879-360eeb61a0bc?auto=format&fit=crop&q=80&w=600',
        standar: [
          `Lokasi Penyimpanan: ${lokasi}`,
          'Dimensi komponen eksternal harus memenuhi lembar spesifikasi cetak.',
          'Bebas dari kontaminasi zat asing, oli abrasif, dan deformasi mikro.',
          'Lolos inspeksi visual menyeluruh di bawah pencahayaan 1000 lux.'
        ]
      }
    });
  };

  return (
    <View style={styles.container}>
      <Text style={styles.title}>Form Tambah Barang Baru</Text>

      <Text style={styles.label}>Nama Barang</Text>
      <TextInput
        style={styles.input}
        placeholder="Nama komponen..."
        placeholderTextColor="#94a3b8"
      />
    </View>
  );
}
```

```

        value={nama}
        onChangeText={setNama}
      />

      <Text style={styles.label}>Kode Serial / SKU</Text>
      <TextInput
        style={styles.input}
        placeholder="Contoh: PT-042, BR-015"
        placeholderTextColor="#94a3b8"
        value={kode}
        onChangeText={setKode}
      />

      <Text style={styles.label}>Lokasi Penyimpanan / Rak</Text>
      <TextInput
        style={styles.input}
        placeholder="Contoh: Rak A-1, Gudang Utama"
        placeholderTextColor="#94a3b8"
        value={lokasi}
        onChangeText={setLokasi}
      />

      <TouchableOpacity style={styles.btnSimpan} onPress={handleSimpan}>
        <Text style={styles.btnSimpanText}>SIMPAN BARANG</Text>
      </TouchableOpacity>
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, padding: 20, backgroundColor: '#fff' },
  title: { fontSize: 20, fontWeight: 'bold', marginBottom: 20, color: '#1e293b' },
  label: { fontSize: 12, fontWeight: 'bold', color: '#64748b', marginTop: 15,
marginBottom: 6, textTransform: 'uppercase' },
  input: { backgroundColor: '#f8fafc', borderWidth: 1, borderColor: '#e2e8f0',
padding: 12, borderRadius: 8, fontSize: 16, color: '#1e293b' },
  btnSimpan: { backgroundColor: '#1e293b', padding: 16, borderRadius: 8, alignItem
'center', marginTop: 30 },
  btnSimpanText: { color: '#fff', fontWeight: 'bold', fontSize: 16 },
});

export default TambahScreen;

```

4. Jawaban Kode Mini Proyek (App.js)

```
import React from 'react';
```

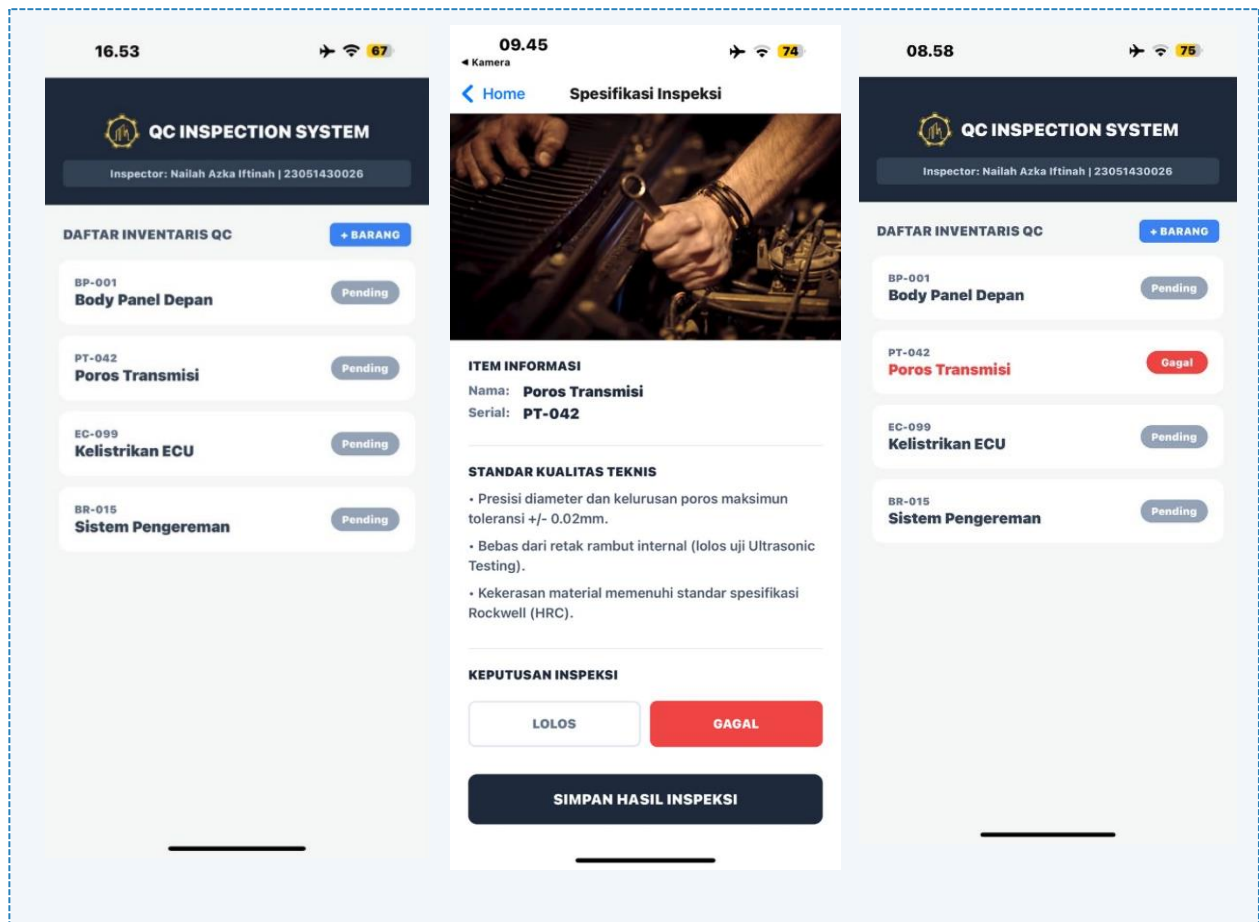
```
import { NavigationContainer } from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';

import HomeScreen from './screens/HomeScreen';
import DetailScreen from './screens/DetailScreen';
import TambahScreen from './screens/TambahScreen';

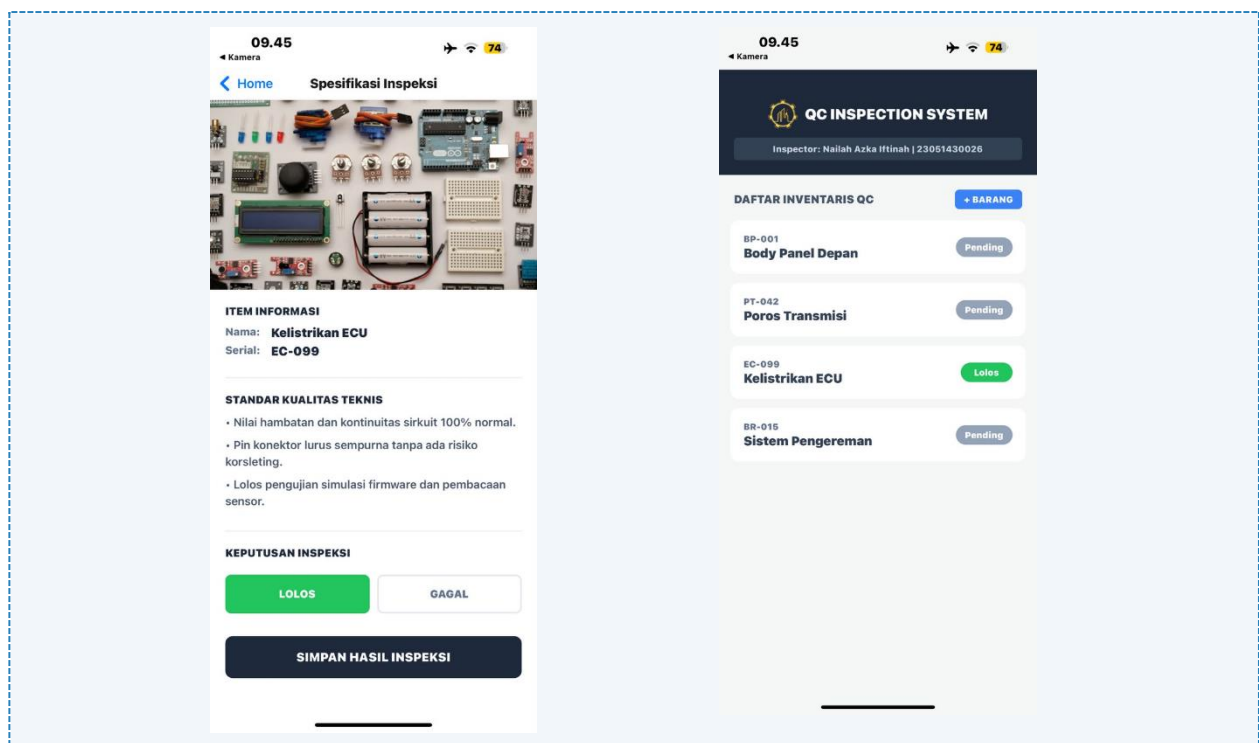
const Stack = createNativeStackNavigator();

export default function App() {
  return (
    <NavigationContainer>
      <Stack.Navigator initialRouteName="Home">
        <Stack.Screen
          name="Home"
          component={HomeScreen}
          options={{ headerShown: false }}
        />
        <Stack.Screen
          name="Detail"
          component={DetailScreen}
          options={{ title: 'Spesifikasi Inspeksi' }}
        />
        <Stack.Screen
          name="Tambah"
          component={TambahScreen}
          options={{ title: 'Tambah Data Inventaris' }}
        />
      </Stack.Navigator>
    </NavigationContainer>
  );
}
```

Screenshot Hasil:



Gambar F.4 – Hasil Proyek Mini



Gambar F.4 – Hasil Proyek Mini

G. PEMBAHASAN

Analisis hasil praktikum secara kritis. Jawab pertanyaan berikut dengan lengkap dan mendalam (bukan hanya "ya/tidak").

G.1 Analisis Kesesuaian Hasil dengan Teori

Hasil praktikum pada pertemuan 15 ini secara keseluruhan telah sesuai dengan teori yang dipelajari mengenai arsitektur navigasi multi-halaman dan pengelolaan data dinamis pada aplikasi mobile menggunakan React Native. Dari proses konfigurasi Stack Navigator, mekanisme pengiriman data parameter antar layar (*passing data params*), hingga penerapan logika pengkondisian (*conditional rendering*), seluruh konsep lanjutan React Native dapat diterapkan secara langsung dalam pembuatan sistem manajemen inventori gudang dan aplikasi inspeksi Quality Control (QC) sederhana. Praktikum ini menunjukkan bahwa React Native mampu digunakan untuk membangun alur kerja (*workflow*) aplikasi mobile yang interaktif, terstruktur, dan mendekati implementasi sistem pemantauan digital pada dunia industri.

Pada tahap awal praktikum, dilakukan penyusunan struktur navigasi tumpukan menggunakan library React Navigation melalui komponen `NavigationContainer` dan `createNativeStackNavigator`. Hal ini sesuai dengan teori arsitektur navigasi mobile bahwa perpindahan halaman tidak berbasis pada perubahan tautan URL teks layaknya website, melainkan menggunakan prinsip *Screen Stack* (tumpukan layar). Komponen halaman utama `HomeScreen` bertindak sebagai dasar tumpukan yang menampilkan daftar inventori gudang secara dinamis, sementara halaman `DetailScreen` dan `TambahScreen` diposisikan di atas tumpukan layar lama ketika diakses menggunakan fungsi `navigation.navigate()`. Saat pengguna menekan tombol kembali (*back*), pemanggilan fungsi `navigation.goBack()` secara otomatis membuang (*pop*) layar teratas dari memori perangkat. Penerapan komponen ini membuktikan bahwa alur perjalanan pengguna (*user journey*) dalam React Native dapat dikelola secara modular, responsif, serta efisien dalam penggunaan konsumsi memori gawai.

Selanjutnya, mekanisme oper data antarlayar lewat properti `route.params` juga telah berjalan sesuai dengan teori integrasi data (*data interchange*) pada pengembangan perangkat lunak lintas platform. Ketika operator memilih salah satu item material pada komponen `FlatList` di halaman utama, paket data berupa objek JavaScript dikirimkan ke halaman detail, lalu dibuka menggunakan teknik *destructuring* untuk memuat informasi nama barang, jumlah stok, hingga informasi spesifik Lokasi Penyimpanan (Rak/Gudang) secara presisi. Kesesuaian ini diperkuat dengan penerapan konsep *Conditional Rendering* pada halaman detail, di mana aplikasi membaca kondisi variabel data stok barang; jika stok bernilai nol (`$ == 0$`), program secara otomatis memicu kemunculan tombol "Request Stok Darurat".

Dalam teori Teknik Industri, logika ini merepresentasikan implementasi nyata dari sistem tarik (*Pull System*) berprinsip *Just-In-Time* (JIT) guna mencegah terjadinya keterlambatan pemenuhan material produksi yang berisiko memicu waktu henti mesin (*production downtime*).

Pada bagian proyek mini, implementasi sinkronisasi data balik untuk aplikasi inspeksi Quality Control (QC) menunjukkan penerapan teori *State Management* lokal yang digabungkan dengan sistem kontrol visual (*Visual Control System*). Ketika inspektur menetapkan status kelayakan produk menjadi "Gagal" (*Defect*) di halaman detail lalu kembali ke halaman beranda, modifikasi *state* tersebut langsung merespons dengan mengubah warna teks komponen barang di `HomeScreen` menjadi warna merah. Pendekatan ini menunjukkan penerapan prinsip *Poka-Yoke* digital (pencegah kesalahan) dan sistem *Andon* visual di era industri modern. Informasi anomali mutu atau cacat produk disajikan secara mencolok untuk memberikan sinyal peringatan dini (*early warning*) bagi operator pergudangan agar segera mengisolasi produk rusak tersebut agar tidak terdistribusi ke proses manufaktur berikutnya atau sampai ke tangan konsumen.

Secara keseluruhan, hasil praktikum menunjukkan bahwa konsep lanjutan React Native seperti arsitektur navigasi tumpukan, oper data dinamis, *conditional rendering*, manipulasi *state*, serta kendali visual berbasis data riil telah diterapkan dengan baik dalam satu ekosistem aplikasi terpadu. Aplikasi yang dihasilkan mampu menyajikan alur pemantauan logistik gudang serta pengawasan mutu produk secara lebih menarik, interaktif, dan mudah dipahami oleh pengguna. Dengan demikian, hasil praktikum telah sesuai dengan tujuan pembelajaran pada pertemuan 15, yaitu memahami penerapan navigasi multi-halaman dan manajemen data React Native dalam membangun sistem monitoring *Quality Control* dan inventori industri modern yang andal serta minim dari risiko kesalahan manusia (*human error*).

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Terjadi <i>error code conflict</i> saat melakukan <i>git push</i> ke GitHub karena folder proyek Expo secara otomatis membuat sub-repository <i>.git</i> internal tersendiri di dalamnya (folder putih tidak bisa diklik di GitHub).	Menghapus file <i>.git</i> tersembunyi di dalam folder Expo menggunakan perintah <code>Remove-Item -Recurse -Force .git</code> melalui PowerShell, melakukan pembersihan <i>cache</i> git, lalu melakukan proses <i>add</i> , <i>commit</i> , dan <i>push</i> ulang dari <i>root folder</i> utama.

G.3 Kaitan dengan Penerapan di Industri

Materi praktikum pada pertemuan 15 ini sangat berkaitan dengan kebutuhan di bidang Teknik Industri, khususnya dalam pengembangan aplikasi monitoring berbasis mobile yang digunakan untuk memantau alur logistik pergudangan dan proses pengendalian kualitas secara lebih dinamis, cepat, dan praktis. Jika pada materi sebelumnya lebih banyak membahas pembuatan dasar tampilan antarmuka statis satu halaman, pada pertemuan ini fokus berpindah ke arsitektur multi-halaman menggunakan React Navigation. Hal ini relevan dengan kondisi industri saat ini, di mana sistem pelaporan deviasi mutu barang dan mutasi material dituntut untuk bergerak fleksibel secara real-time mengikuti mobilitas tinggi supervisor, inspektur QC, maupun operator langsung di lantai produksi.

Pembuatan aplikasi sistem manajemen inventori gudang dan inspeksi Quality Control (QC) pada praktikum ini mencerminkan penerapan sistem pengawasan di industri nyata. Dalam dunia manufaktur modern, ketertelusuran produk (product traceability) dan kecepatan pengambilan keputusan mutu merupakan aspek kritical untuk meminimalkan produk cacat (defect rate). Dengan adanya fitur navigasi Screen Stack dan mekanisme pengiriman data (passing data params), operator dapat memantau detail spesifikasi barang, jumlah stok aktual, hingga informasi spesifik Lokasi Penyimpanan (Rak/Gudang) tanpa risiko salah ketik atau data tertukar. Selain itu, penggunaan indikator visual kontrol seperti perubahan warna teks menjadi merah saat produk dinyatakan "Gagal" sangat relevan dengan sistem Poka-Yoke (pencegah kesalahan) dan prinsip Andon di pabrik. Konsep ini membantu operator mengenali anomali atau kondisi kritis dengan lebih cepat, sehingga proses isolasi produk cacat dapat segera dilakukan sebelum barang terdistribusi ke proses berikutnya atau sampai ke tangan konsumen.

Penerapan logika pengkondisian (conditional rendering) untuk memunculkan tombol "Request Stok Darurat" saat persediaan habis bernilai nol juga memiliki kaitan erat dengan strategi kontrol inventori pada industri modern. Sistem monitoring industri dituntut bersifat prediktif dan mampu memicu tindakan korektif secara instan guna menghindari berhentinya lini produksi (production downtime) akibat keterlambatan pasokan material. Logika ini secara langsung mengadopsi prinsip sistem tarik (Pull System) dalam manajemen Just-In-Time (JIT). Di sisi lain, kemampuan React Native dalam mendukung cross-platform development memberikan efisiensi tinggi bagi perusahaan dalam mengurangi biaya investasi perangkat lunak (software development cost), karena satu aplikasi yang dikembangkan dapat langsung dioperasikan pada smartphone operasional milik operator hingga tablet industri yang dipasang di area rak penyimpanan gudang.

Secara keseluruhan, praktikum pada pertemuan 15 ini memberikan gambaran nyata mengenai bagaimana aplikasi mobile dikembangkan untuk mendukung aktivitas manajemen rantai pasok dan kendali mutu industri. Mulai dari penyusunan alur navigasi antar layar, integrasi oper data yang aman, hingga visualisasi kondisi darurat berbasis data riil, seluruh konsep yang dipelajari sudah sangat relevan dengan kebutuhan industri modern era Manufaktur Cerdas (Smart Manufacturing). Meskipun aplikasi yang dibuat masih dalam skala simulasi, konsep dasarnya sudah mendekati sistem eksekusi manufaktur (Manufacturing Execution System) berbasis mobile yang digunakan dalam digitalisasi operasional industri nyata.

H. KESIMPULAN

Tuliskan 3–5 poin kesimpulan yang merangkum hal-hal yang dipelajari dan dicapai pada praktikum modul ini. Kesimpulan harus spesifik, bukan bersifat umum.

1	Mahasiswa mampu memahami cara membuat aplikasi yang memiliki banyak halaman menggunakan React Navigation, di mana halaman baru akan ditumpuk di atas halaman lama (Screen Stack) dan bisa kembali dengan lancar.
2	Mahasiswa mampu mengirim data dari satu halaman ke halaman lain (<i>passing data params</i>), sehingga saat menyentuh salah satu barang di halaman utama, detail seperti nama, jumlah stok, dan lokasi rak gudang bisa muncul secara akurat di halaman detail.
3	Mahasiswa memahami cara mengatur tampilan tombol otomatis (<i>conditional rendering</i>), contohnya memunculkan tombol "Request Stok Darurat" hanya ketika stok barang habis atau bernilai nol untuk mencegah berhentinya proses produksi.
4	Mahasiswa mampu membuat sistem peringatan visual, di mana warna teks barang di halaman utama otomatis berubah menjadi merah ketika inspektur memilih status "Gagal" (<i>Defect</i>) pada saat melakukan cek kualitas (<i>Quality Control</i>).
5	Mahasiswa mampu membuat aplikasi inventori dan cek kualitas barang berbasis HP yang rapi, mudah digunakan oleh operator, dan siap mendukung sistem kerja digital di lingkungan industri modern.

I. DAFTAR PUSTAKA

Tuliskan semua sumber referensi yang digunakan dalam format APA 7th Edition. Minimal 3 referensi yang relevan (termasuk Manual Praktikum dan sumber online resmi seperti MDN, Reactjs.org, dll).

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	Karim, M. A., & Adriansyah, A. R. (2022). Analisis dan Perancangan Aplikasi Mobile untuk Donasi menggunakan Metode Hybrid berbasis React Native. Jurnal Informatika Terpadu, 8(1). https://doi.org/10.54914/jit.v8i1.394
[3]	Wibawa, Y. E., & Naufal, M. (2023). Pembangunan Perangkat Lunak Komunitas Media Musik dengan Framework React Native dan Firebase Berbasis Android dan IOS. Jurnal SISKOM-KB (Sistem Komputer dan Kecerdasan Buatan), 6(2). https://doi.org/10.47970/siskom-kb.v6i2.515
[4]	Bakhtiar, M., & Chotijah, U. (2025). Pengembangan Computerized Maintenance Management System (CMMS) Berbasis Mobile dengan React Native sebagai Solusi Monitoring Asset dan Work Order. Jurnal Nasional Komputasi dan Teknologi Informasi (JNKTI), 8(6).
[5]	Prayogo, M. T., & Waluyo, A. F. (2024). Inovasi Dalam Pengembangan Aplikasi Mobile Untuk Monitoring Pembayaran Wifi. INTECOMS: Journal of Information Technology and Computer Science, 7(6). https://doi.org/10.31539/intecom.v7i6.12758

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Diperiksa oleh Asisten / Dosen
 Nailah Azka Iftinah NIM:23051430026	 Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____